# Week 3: Operating and Maintaining the Urban Data Platform

Your data platform has now been deployed by the municipality. Every day, new datasets arrive, existing records are corrected, schemas evolve, and analysts continue to execute analytical queries. The platform must continue operating while preserving correctness, reliability, and performance.

Your task is to transform your prototype into a production-ready data platform that supports incremental updates, evolving datasets, monitoring, and long-term maintainability.

## Task 1. Generating Incremental Update Datasets

The municipality has released a second version of the datasets. Since only the initial release is provided, your first task is to simulate realistic incremental data releases for each dataset. These updates will be used to evaluate your incremental processing pipeline. Create one update file for each dataset. The update files should contain only the new or modified records, not the complete dataset. All generated records must be syntactically valid and follow the schema of the original dataset, except where schema evolution is explicitly introduced.

- **Taxi Trips:** Create a Parquet file containing 5–10% new taxi trips with timestamps occurring after the latest trip in the original dataset. The new trips should resemble the original data (e.g., similar pickup and dropoff locations, trip distances, and fare amounts). Include 1–2% duplicate trips copied from the original dataset.
- **Weather:** Create a CSV file containing new hourly weather observations for the period immediately following the original dataset. Generate realistic values for the existing attributes and add one new column, `humidity`, to simulate schema evolution. The `humidity` column should contain realistic relative humidity values (typically between 20% and 100%) represented as numeric percentages.
- **Air Quality:** Create a CSV file containing new hourly air-quality observations for the period immediately following the original dataset. Generate realistic values for the existing attributes and add one new column, `aqi`, to simulate schema evolution. The `aqi` column should contain realistic Air Quality Index (AQI) values (typically between 0 and 500), where lower values indicate better air quality.

Implement an incremental update pipeline that:

- inserts new records,
- ignores duplicate records,
- preserves unchanged records,
- supports schema evolution,
- updates the corresponding Delta tables without rebuilding the platform.

Document the changes introduced in each update file, including:

- the number of new records,
- the number of duplicate records (where applicable),
- the schema changes introduced.

## Task 2. Maintain Analytical Consistency

The analytical queries and data products developed in Week 2 should continue to produce valid and up-to-date results after new data arrives and schemas evolve. Extend your platform so that it:

- refreshes only the analytical data products affected by new data or schema changes,
- supports schema evolution,
- preserves compatibility with existing analytical queries whenever possible,
- minimizes unnecessary recomputation.

Discuss

- Which analytical data products can be refreshed incrementally?
- Which require complete recomputation?
- Which schema changes can be handled automatically?
- Which schema changes require manual intervention?
- How does your design reduce unnecessary computation?

## Task 3. Build a Platform Monitoring System

A production platform should continuously report its operational status. Implement a monitoring component that automatically records metadata about every pipeline execution. At a minimum, record:

- pipeline execution time,
- processed records,
- inserted records,
- rejected records,
- schema version,
- validation failures.

Store this information in one or more Delta tables. Implement Spark SQL queries that answer questions such as:

- Which dataset fails validation most frequently?
- Which dataset requires the longest processing time?
- How many records were rejected during each execution?
- How has processing time changed over multiple executions?

Discuss

- Which operational metrics are most useful?
- How could this information support debugging and system maintenance?
- What additional monitoring information would be valuable in a production system?

## Task 4. Extend the Data Validation Framework

Extend the validation framework developed in Week 1. Your framework should automatically detect:

- duplicate records,
- invalid attribute values,
- missing reference records,
- unexpected or unsupported schema changes,
- incomplete records.

Invalid records should:

- not interrupt processing,
- be isolated,
- be reported,
- be excluded from analytical data products.

The framework should allow new validation rules to be added with minimal code changes.

Discuss

- Which validation rules are generic and can be applied across all datasets?
- Which validation rules are dataset-specific?
- How can new validation rules be added without modifying the core validation framework?

## Task 5. Evaluate the Platform

Evaluate the production readiness of your platform. Measure:

- incremental update time,
- analytical refresh time,
- storage overhead introduced by the updated platform,
- validation overhead (additional execution time introduced by validation),
- monitoring overhead (additional execution time introduced by monitoring).

Discuss

- Which design decisions from Week 1 simplified maintenance?

- Which components required the largest modifications?
- How well does the platform support future datasets?
- If you redesigned the platform today, what would you change?

Support your conclusions using measurements and examples from your implementation.

## Deliverables

Each group should submit

- **Source code**, i.e., the complete Spark project, including the incremental update pipeline, schema evolution support, analytical data refresh, monitoring framework, extended validation framework, evaluation experiments, and configuration files.
- **A 3–5 page design report**, containing the incremental processing strategy, analytical consistency strategy, monitoring architecture, validation framework, engineering decisions, and trade-offs.
- **A short evaluation report**, including the incremental update performance, analytical refresh performance, validation statistics, monitoring results, maintainability analysis, and discussion of scalability.
- **A README** describing how to process incremental updates, handle schema evolution, perform monitoring, generate validation reports, and reproduce the evaluation experiments.